

Interview Q&A;

Automotive / AAOS interviews: sourced questions + answer bank

"Interviewers don't want definitions. They want to hear that you've been in the car."

- part A: real reports with company, date & source
- part B: the AAOS topic bank
- part C: system design + your 60-second pitch

A Head First-style learning guide that goes with the **Automotive Academy** sample app (Android Automotive OS, Kotlin, Jetpack Compose). Generated September 2026.

What's in this book

Read this first: how the questions were collected	3
A Questions reported by candidates (with company, date, source)	3
Model answers (with a car twist)	5
B AAOS topic bank	7
Platform & architecture	7
VHAL & properties	8
Driver distraction & UI	8
Media & audio	9
Power, OTA, multi-user	9
Vehicle networks & security	9
Car App Library	10
C System design rounds	10
Design a media streaming app for AAOS (e.g. for Harman's OEM customer)	10
Design the HVAC system app	10
Design an OTA client for the head unit	11
D Talk about this project	11
E Behavioural questions for Tier-1 / OEM roles	12

Read this first: how the questions were collected

Part A lists questions that candidates **publicly reported** on AmbitionBox, GeeksforGeeks and Medium. They were retrieved on 27 Sep 2026. For each one the book gives the company, the role, the **date shown on the source** (usually when the report was posted, which can be after the interview) and the source link. These are candidate reports, not official question banks.

WATCH IT!

An honest observation from the sources: the **publicly posted** Harman, Bosch, KPIT, Tata Elxsi, Mercedes-Benz R&D; and LTTS reports mostly contain **Android fundamentals, Kotlin, design patterns, C/embedded and DSA**. Deep AAOS questions (VHAL, CarService, UX restrictions) are asked in automotive roles but are rarely written up with dates.

So Part B is a **topic bank** covering those AAOS areas. It is **not** attributed to any company or date, because inventing that would mislead you.

A Questions reported by candidates (with company, date, source)

Company	Role	Date on source	Reported questions	Source
HARMAN	Android Developer	11 Jul 2026	• What are the key components of the MVVM architecture in Android, and how do they interact? • Describe the Android application lifecycle. How do activities transition between states? • What are SOLID principles? • What is the Android Framework? • Write a Builder class. • Explain the purpose of the Room persistence library. • 'More in-depth on Android internals'	AmbitionBox ambitionbox.com/interviews/harman-interview-questions/android-developer
HARMAN	Embedded Automotive Developer	12 Jul 2025	• Smart pointers, bitwise operators • Multithreading, STL	AmbitionBox ambitionbox.com/interviews/harman-interview-questions/embedded-automotive-developer
HARMAN	Android Developer (Round 1)	30 Dec 2024	• Explain activity lifecycle • Explain MVVM architecture	AmbitionBox ambitionbox.com/interviews/harman-interview-questions/android-developer
HARMAN	Android Developer (Round 2)	7 Jan 2023	• Explain Coroutines, RxJava • Kotlin: lateinit, val, var, extension function, data class	AmbitionBox ambitionbox.com/interviews/harman-interview-questions/android-developer

Company	Role	Date on source	Reported questions	Source
HARMAN International	Android role (6 yrs exp.)	23 Aug 2019	• Which lifecycle methods are called when the system kills the process? • What is a Handler and how does it work internally? • Is a bound service synchronous? Have you used AIDL? • How does a memory leak happen? • How is ViewModel implemented internally? • Difference between IntentService and Service; bound vs started service • SharedPreferences apply vs commit • Activity launch modes	GeeksforGeeks geeksforgeeks.org/interview-experiences/harman-international-interview-experience-for-android-role/
Bosch	Android Developer	25 Nov 2025	• Why does Bosch prefer Kotlin over Java? • Coroutines vs Threads: when do you choose which?	Medium (Mobile App Developer) medium.com/@mobileappdeveloper.koti/bosch-android-developer-interview-experience-d6d15cfe207c
Mercedes-Benz R&D; India	Android Developer	29 Jun 2022	• Advantage of MVVM over MVP • What is a foreground service? Foreground vs background services • How does ViewModel work internally? • Internal working of LiveData • Coroutines and their scopes • Fragment lifecycle for add vs replace • Implement the Observer / Builder pattern	Medium (Piyush Sinha) medium.com/@piyush.sinha12345/mercedes-benz-android-developer-interview-experience-9b727087e0a9
L&T; Technology Services	Android Framework Developer	5 Mar 2025	• Explain Android architecture in brief • Explain the Android boot process	AmbitionBox ambitionbox.com/interviews/l-and-t-technology-services-interview-questions/android-framework-developer
Tata Elxsi	Android Developer	11 Nov 2024	• Java program on strings, threads • Multithreading, collections • Git commands, Linux commands	AmbitionBox ambitionbox.com/interviews/tata-elxsi-interview-questions/android-developer
KPIT Technologies	Android Developer	23 Mar 2023	• Difference between interface and abstract class • What is a singleton class?	AmbitionBox ambitionbox.com/interviews/kpit-technologies-interview-questions/android-developer
Visteon	Embedded / software (GfG write-up)	7 May 2024	• Setting and clearing a bit (optimised) • Difference between C and embedded C • Structure vs union • Microprocessor vs microcontroller; ARM vs x86 • Compilation process in C • Find a loop in a linked list	GeeksforGeeks geeksforgeeks.org/interview-experiences/visteon-corporation-interview-experience/
Visteon	Software Developer	19 Feb 2024	• C and security, protocol, middleware	AmbitionBox ambitionbox.com/interviews/visteon-interview-questions/android-developer

Model answers (with a car twist)

tip: always end a fundamentals answer with 'in a car this matters because...'

Q. MVVM components and interaction

View (Activity/Fragment/Composable) observes state from the ViewModel and forwards user events. ViewModel holds UI state (StateFlow/LiveData), survives configuration changes, calls the Model. Model = repositories + data sources (Room, network, **VHAL** in a car). Data flows up as state, events flow down: unidirectional data flow. In this project: HvacScreen → VehicleViewModel → VehicleRepository → CarApiVehicleDataSource → CarPropertyManager.

Q. Activity lifecycle

onCreate → onStart → onResume (interactive) → onPause → onStop → onDestroy; onRestart when coming back from stopped. **Car twist:** AAOS head units change displays/users, and ActivityBlockingActivity covers non-DO activities while driving, which pauses your activity. Save state in onStop / ViewModel + SavedStateHandle.

Q. Which callbacks run when the system kills the process?

None are guaranteed. onSaveInstanceState runs after onStop on API 28+ (before onStop on older versions); a low-memory kill of a stopped process calls nothing. So persist important data early (onStop) and restore from SavedStateHandle / disk.

Q. SOLID

S: single responsibility (VehiclePhysics only does physics). O: open for extension, closed for modification (add a new VehicleDataSource without editing screens). L: Liskov: any VehicleDataSource can replace another. I: interface segregation: small interfaces. D: dependency inversion: screens depend on the VehicleDataSource abstraction, not on CarPropertyManager.

Q. What is the Android Framework? / Android architecture

Layers: Linux kernel → HAL (hardware interfaces; for cars the Vehicle HAL) → native libraries + ART → Java framework (ActivityManager, PackageManager, WindowManager... and in AAOS the Car API + CarService) → apps. Apps call framework APIs; system services run in system_server and talk to apps over Binder.

Q. Android boot process

Boot ROM → bootloader (verified boot) → kernel → init (reads init.rc, starts ueventd, servicemanager, HALs, surfaceflinger) → Zygote (preloads classes) → system_server (starts system services) → CarService on AAOS → BOOT_COMPLETED → launcher. On AAOS, EVS and early audio start before Zygote so the rear camera works quickly.

Q. Builder class

Use when an object has many optional parameters. In Kotlin, named/default args often replace it, but Java interop and Android APIs (AudioFocusRequest.Builder, Row.Builder in Car App Library) use it.

Q. Room

SQLite abstraction: @Entity, @Dao, @Database; compile-time SQL verification; returns Flow for observable queries; migrations. In a car: store recent destinations or media history **per Android user**.

Q. Coroutines vs Threads; RxJava

Threads are OS resources (~1 MB stack), expensive to create and block. Coroutines are lightweight, suspend instead of blocking, and use structured concurrency (scopes cancel children). Use threads/dispatchers underneath (Dispatchers.IO/Default); use coroutines for app logic. RxJava is a reactive stream library; Flow is the Kotlin-native equivalent. This app's simulator is a coroutine loop; CarService callbacks are bridged into StateFlow.

Q. Coroutine scopes

GlobalScope (avoid), viewModelScope (cancelled in onCleared), lifecycleScope, custom CoroutineScope(SupervisorJob()+Dispatcher) for repositories (see VehicleRepository). SupervisorJob: one child failing doesn't cancel siblings.

Q. Kotlin: lateinit, val, var, extension functions, data class

val = read-only reference; var = mutable; lateinit var = non-null var initialised later (not for primitives; check ::x.isInitialized). Extension functions add functions to a type without inheritance (resolved statically). data class generates equals/hashCode/toString/copy/componentN (VehicleState uses copy for immutable updates).

Q. Why Kotlin over Java?

Null safety, coroutines, concise syntax, data/sealed classes, extension functions, first-class Compose, full Java interop, Google's recommended language. For car apps: less boilerplate in state machines and safer null handling of vehicle properties that may be unavailable.

Q. Handler internals

A Looper runs a MessageQueue on a thread; Handler posts Messages/Runnables to that queue and handles them on the Looper's thread. The main thread has a Looper by default. Car API callbacks can be delivered on a Handler you pass to Car.createCar.

Q. Bound service synchronous? AIDL?

Binding is asynchronous (onServiceConnected later). Calls through an AIDL interface are synchronous by default (the caller blocks) unless declared oneway. The whole Car API is AIDL over Binder to CarService, and VHAL itself is an AIDL HAL.

Q. Memory leaks

Holding a reference to an Activity/Context beyond its life: static fields, singletons, inner classes/anonymous listeners, unregistered callbacks. **Car example:** registering a CarPropertyEventCallback in an Activity and never calling unregisterCallback keeps the Activity alive.

Q. ViewModel internals

ViewModelStoreOwner (Activity/Fragment/NavBackStackEntry) owns a ViewModelStore (a map). ViewModelProvider creates or returns the instance by key. The store is retained across configuration changes (via NonConfigurationInstances) and cleared (onCleared) when the owner is finished.

Q. LiveData internals

Holds a value + version; observers are wrapped with a LifecycleBoundObserver and only receive updates when STARTED/RESUMED; they're removed on DESTROYED. setValue on main thread, postValue from any thread.

Q. IntentService vs Service; bound vs started; foreground

IntentService (deprecated) ran work on a worker thread and stopped itself; use WorkManager/coroutines. Started service runs until stopSelf; bound service lives while clients are bound. Foreground service shows a notification and has a type (e.g. mediaPlayback), which a car media app uses while playing.

Q. apply vs commit; launch modes

commit writes synchronously and returns a boolean; apply writes async. Launch modes: standard, singleTop, singleTask, singleInstance (+ singleInstancePerTask).

Q. MVVM vs MVP

MVP presenter holds a View reference (interfaces, manual lifecycle handling, leak risk). MVVM ViewModel exposes observable state and knows nothing about the View; lifecycle-aware and easier to test.

Q. Interface vs abstract class; singleton

Interface: no state, multiple inheritance of type, default methods allowed. Abstract class: can hold state and constructors, single inheritance. Singleton in Kotlin: object declaration (thread-safe lazy init by the JVM).

Q. C / embedded (Visteon, Harman embedded)

Set bit: $x |= (1u << n)$; clear: $x \&= \sim(1u << n)$; toggle: $x \wedge= (1u << n)$. struct = members have separate memory (size $\geq$ sum, with padding); union = members share memory (size = largest). Embedded C adds fixed-point, memory-mapped I/O, volatile, ISR constraints. Compilation: preprocess $\rightarrow$ compile $\rightarrow$ assemble $\rightarrow$ link. Loop in a list: Floyd's slow/fast pointers. Smart pointers: `unique_ptr` (sole ownership), `shared_ptr` (ref count), `weak_ptr` (breaks cycles).

B AAOS topic bank

not tied to a specific company or date: these are the areas to master

Difficulty tags: **basic** (any Android dev moving to automotive), **intermediate** (2-5 years), **advanced** (platform/framework roles at Tier-1s and OEMs).

Platform & architecture

Q. What is AAOS and how does it differ from Android Auto? [basic]

AAOS is Android running natively on the head unit with access to vehicle data via VHAL; Android Auto runs on the phone and projects to the car display. Same media and Car App Library APIs work on both.

Q. Walk me through the AAOS stack. [basic]

Apps $\rightarrow$ Car API managers (android.car) $\rightarrow$ CarService (binder, permissions, policies) $\rightarrow$ VHAL (AIDL IVehicle) $\rightarrow$ vehicle MCU/gateway $\rightarrow$ CAN/LIN/Ethernet $\rightarrow$ ECUs. Mention CarService restart handling.

Q. What is CarService? [basic]

The privileged system component that implements the Car API: property, UX restrictions, audio, power, occupant zones, input, watchdog. It sits between apps and VHAL, enforces car permissions and multiplexes subscriptions.

Q. What happens when CarService crashes? [intermediate]

Apps using `Car.createCar` with a lifecycle listener get `ready=false`; managers are invalid. When it restarts the listener fires with `ready=true`; re-fetch managers and re-register callbacks.

Q. What is GAS? [basic]

Google Automotive Services: Google Maps, Assistant, Play Store and other Google apps licensed to OEMs ('Google built-in'). AAOS can ship without GAS.

Q. Why does AAOS have a headless system user? [intermediate]

User 0 runs system services only; drivers use secondary users so profiles can switch (driver change, guest) without rebooting, and data stays separated.

Q. How is the AAOS boot optimised for cars? [advanced]

Early native services (EVS camera, early audio chimes), suspend-to-RAM instead of cold boot, deferred non-critical services, pre-created users, and boot-time budgets from the OEM.

VHAL & properties

Q. What is a VHAL property? Describe its attributes. [basic]

32-bit ID (group|area|type|id), value type, area type & area IDs, access (READ/WRITE/READ_WRITE), change mode (STATIC/ON_CHANGE/CONTINUOUS), min/max sample rate, required permissions.

Q. Decode 0x11600207. [intermediate]

SYSTEM (0x1), GLOBAL (0x1), FLOAT (0x60), id 0x0207 → PERF_VEHICLE_SPEED in m/s.

Q. ON_CHANGE vs CONTINUOUS? [basic]

ON_CHANGE fires only when the value changes (gear, door). CONTINUOUS is sampled at a requested rate within config limits (speed, rpm).

Q. How do you add an OEM-specific property? [advanced]

Define a VENDOR group ID (0x2...), implement it in the VHAL, add SELinux/permission (often CAR_VENDOR_EXTENSION or custom), expose to privileged apps; document in the OEM's property list.

Q. What are area IDs? Give an HVAC example. [intermediate]

For area properties the same ID exists per area. HVAC_TEMPERATURE_SET with area ROW_1_LEFT (0x1) and ROW_1_RIGHT (0x4), or OEM groupings like 0x31. Read areaIds from CarPropertyConfig.

Q. AIDL vs HIDL VHAL? [intermediate]

HIDL (2.0) was used until Android 12; Android 13 introduced the stable AIDL VHAL (IVehicle) with better versioning and batch APIs. New work targets AIDL.

Q. How do you test VHAL-dependent code without hardware? [intermediate]

Emulator car data controls, reference/fake VHAL, 'cmd car_service inject-vhal-event', and abstraction layers faked in unit tests.

Q. A property returns STATUS_UNAVAILABLE. What do you do? [intermediate]

Treat as a normal state (e.g. HVAC off, sensor not ready). Show a disabled/placeholder UI; don't crash; listen for the next event.

Driver distraction & UI

Q. How does an app know it must restrict UI while driving? [basic]

Register with CarUxRestrictionsManager; react to flags (NO_VIDEO, NO_KEYBOARD, LIMIT_CONTENT...) and limits. Don't read speed yourself.

Q. What does distractionOptimized mean? [intermediate]

Manifest meta-data declaring an activity follows DO guidelines and may run while moving; otherwise it's blocked by ActivityBlockingActivity. Honoured for system/pre-installed apps.

Q. Who configures the restrictions? [intermediate]

The OEM, in car_ux_restrictions_map.xml per driving state (and per display).

Q. Design guidelines for car UI? [basic]

Large touch targets (~76dp), high contrast, day/night themes, glanceable (≤2 s glances), minimal text, voice alternatives, rotary/knob support, consistent with OEM theming (RROs).

Q. What are RROs in AAOS? [advanced]

Runtime Resource Overlays: OEMs restyle system UI and car-ui-lib components without changing app code.

Media & audio

Q. How does a media app integrate with the car? [basic]

MediaBrowserService (tree) + MediaSession (control); the car's media UI renders it. Validate callers in onGetRoot, set content-style hints, declare PlaybackState actions, support search.

Q. Explain audio focus in AAOS. [intermediate]

Usage maps to a context; CarAudioService uses an interaction matrix: exclusive, concurrent (duck) or reject. Apps request focus with correct AudioAttributes and handle loss/duck callbacks.

Q. What is an audio zone? [intermediate]

An independent set of output devices and volume groups (e.g. rear-seat headphones) mapped to an occupant zone.

Q. Why fixed volume on AAOS? [advanced]

Gain is applied by the external amplifier/DSP per bus; Android outputs at fixed volume.

Power, OTA, multi-user

Q. Describe car power states. [intermediate]

ON, SHUTDOWN_PREPARE (save state, Garage Mode), WAIT_FOR_VHAL, suspend-to-RAM/disk, OFF; driven by the vehicle MCU via AP_POWER_STATE_REQ.

Q. What is Garage Mode? [basic]

After ignition off, AAOS stays up briefly to run idle-constrained JobScheduler jobs (updates, sync) before suspending.

Q. How are OTA updates made safe? [basic]

Signed packages, A/B slots, install while driving, reboot when parked with consent, verified boot + rollback, R156-compliant SUMS.

Q. What is MUMD? [advanced]

Multi-user multi-display (Android 14+): concurrent users on passenger/rear displays, each with its own apps and audio zone.

Vehicle networks & security

Q. How does CAN arbitration work? [basic]

Wired-AND bus; nodes send IDs MSB-first; dominant 0 wins; losers back off without corrupting the winner's frame. Lower ID = higher priority.

Q. CAN vs CAN FD vs Ethernet? [basic]

CAN: 8 bytes, ≤1 Mbps. CAN FD: 64 bytes, faster data phase. Automotive Ethernet: 100M-multi-gig for cameras/backbone, SOME/IP, DoIP.

Q. What is a DBC file? [basic]

The communication matrix describing frames (ID, DLC, sender) and signals (start bit, length, endianness, factor, offset, unit).

Q. Name some UDS services. [intermediate]

0x10 session, 0x11 reset, 0x19 read DTC, 0x22 read DID, 0x27 security access, 0x29 authentication, 0x2E write DID, 0x31 routine control, 0x34/0x36/0x37 download.

Q. What is SecOC? [intermediate]

AUTOSAR message authentication: truncated MAC + freshness value appended to PDUs to prevent spoofing, tampering and replay.

Q. R155 vs R156 vs ISO/SAE 21434? [intermediate]

R155: cybersecurity management system (regulation). R156: software update management system. ISO/SAE 21434: the engineering standard describing how (TARA, etc.).

Q. How would you secure a car app's backend token? [basic]

Android Keystore (AES-GCM), per user, never in code/logs; TLS with pinning; short-lived tokens + refresh.

Car App Library

Q. Explain the Car App Library lifecycle. [basic]

Host binds CarAppService → createHostValidator → onCreateSession → Session.onCreateScreen → Screen stack via ScreenManager → onGetTemplate; invalidate() to refresh.

Q. Why templates? [basic]

Host renders them, so they're distraction-optimized, OEM-styled, and consistent across Android Auto & AAOS.

Q. What limits do templates impose? [intermediate]

List length (often 6 while driving), task depth (~5 screens), refresh quotas, content types per category.

C System design rounds

Design a media streaming app for AAOS (e.g. for Harman's OEM customer)

- **Clarify:** GAS or not, offline support, voice, multiple users, rear-seat zone.
- **Architecture:** Media3 MediaLibraryService + ExoPlayer; repository with cache (Room) per user; download worker (WorkManager, runs in Garage Mode).
- **Car integration:** browse tree ≤ 4 levels, content style hints, search (voice), PlaybackState actions, custom actions (like/shuffle), error states with resolution intents (sign-in on phone).
- **Distraction:** no own UI while driving; settings/sign-in only when parked (NO_SETUP).
- **Audio:** USAGE_MEDIA focus, handle duck/transient loss; resume after calls.
- **Security:** validate callers in onGetRoot, tokens in Keystore, per-user data.
- **Testing:** MediaController test harness (like MediaClient.kt), emulator, DHU for Android Auto.

Design the HVAC system app

- Privileged system app (priv-app + allowlist) with CONTROL_CAR_CLIMATE.
- Reads areaIds per property; builds zones dynamically (2-zone, 3-zone, 4-zone cars).
- Optimistic UI + reconcile with ON_CHANGE events; handle HVAC_POWER_ON gating and unavailable status.
- Debounce knob rotations (don't flood VHAL); respect min/max from config.
- Night mode, rotary input, accessibility; unit tests with fake data source; HIL with CANoe.

Design an OTA client for the head unit

- Check-in with backend (vehicle identity + current versions), download over Wi-Fi/cellular policy, resumable.
- Verify signature and hash; apply via update_engine to inactive slot while driving.
- Ask user; install/reboot only when parked (driving state + power policy); Garage Mode or scheduled time.
- Report status for R156 audit; automatic rollback if boot not marked successful; delta payloads.

INTERVIEW TIP

Structure every design answer: clarify → constraints (driving, safety, power, users) → components → data flow → security → testing → trade-offs. Drawing the AAOS stack in the first minute shows domain fluency.

D Talk about this project

Resume line: Built an Android Automotive OS learning app (Kotlin, Jetpack Compose) that reads live vehicle data via CarPropertyManager with a simulator fallback, reacts to CarUxRestrictions, exposes a MediaBrowserService and a Car App Library charging app, and demonstrates CAN arbitration, SecOC, A/B OTA and power state machines with unit tests.

60-second pitch: 'I wanted to understand AAOS end to end, so I built an app where every concept is a live demo. The core is a VehicleDataSource interface with two implementations: a physics simulator for phones and a Car API implementation that subscribes to VHAL properties through CarPropertyManager and re-subscribes when CarService restarts. The UI is Compose with MVVM and StateFlow. For HVAC I look up area IDs from CarPropertyConfig instead of hard-coding them, and handle the SecurityException because CONTROL_CAR_CLIMATE is privileged. I also modelled CAN arbitration, SecOC replay protection and A/B OTA rollback as pure Kotlin state machines with JVM tests.'

SHARPEN YOUR PENCIL

Practise out loud:

1. Explain what happens between turning the HVAC knob and the blend door moving.
2. Explain why your app can read speed but cannot change the temperature on a real car.
3. Explain what your MediaBrowserService does when the steering-wheel 'next' button is pressed.

E Behavioural questions for Tier-1 / OEM roles

- **Working with OEM customers:** requirements change late; talk about clarifying specs and change requests.
- **Quality mindset:** ASPICE processes, code reviews, traceability from requirement to test.
- **Debugging on target:** logcat, dumphsys car_service, bugreports, CAN traces; reproducing on HIL benches.
- **Safety culture:** 'I would rather delay a release than ship something that can distract the driver.'

BULLET POINTS

- Fundamentals (lifecycle, MVVM, coroutines, Kotlin) dominate public reports; know them cold.
- Always add a car-specific angle to fundamental answers.
- Master the AAOS stack, VHAL properties, UX restrictions, media, audio focus.
- Know CAN basics, UDS, SecOC, R155/R156/21434.
- Use this project as your story.